

Internship Daily Documentation

Day 3 — Wednesday, April 29, 2026

Smart Internship & Hiring Portal | Intern: Mageswaran G

Field	Details
Date	Wednesday, April 29, 2026
Intern	Mageswaran G
Project	Smart Internship & Hiring Portal
Mentor	Sandiya
Module	Week 1 — Project Setup & Authentication
Day Goal	Login API, JWT token system, RBAC, Refresh tokens, Production-grade security, Service layer
Achievement	Upgraded from basic to full production-grade authentication system

1. SOD (Start of Day) — Morning Plan

Planned Tasks:

- Login API — POST /api/auth/login
- JWT Token Generation — access token + refresh token
- API Response Structuring — standard format
- Testing — validate credentials, test error handling

All 4 planned tasks completed. Plus major production-grade upgrade done beyond plan.

2. What Was Built Today

2.1 Login API

File: controllers/authController.js + services/authService.js

- User fetched from MongoDB by email
- Password compared using bcrypt.compare via user.comparePassword()
- Invalid credentials return 401 with message: Invalid credentials
- On success: Access Token (15 min) + Refresh Token (7 days) generated
- Refresh token hashed with bcrypt before storing in MongoDB
- Access token returned in response body
- Refresh token stored in HTTP-only cookie (secure — JavaScript cannot read it)

2.2 JWT Middleware — verifyToken

File: middleware/authMiddleware.js

- Checks Authorization header for Bearer token

- Validates token format (must start with Bearer)
- Verifies JWT with JWT_SECRET from .env file
- Attaches decoded user (id, role, email) to req.user
- Returns 401 if token is missing or invalid

2.3 Role-Based Access Control (RBAC)

File: middleware/authMiddleware.js — authorizeRole function

- authorizeRole('admin') blocks candidate from admin routes
- Returns 403 Forbidden for wrong role
- Safety check: req.user must exist before role check
- Tested: candidate token on admin route = 403
- Tested: admin token on admin route = 200

2.4 Refresh Token System

File: utils/generateToken.js, services/authService.js

- generateAccessToken() — 15 minute expiry
- generateRefreshToken() — 7 day expiry
- Refresh token hashed with bcrypt before DB storage
- On refresh: bcrypt.compare validates old token
- Token rotation: new refresh token issued on every /refresh call
- Token reuse detection: if invalid token, all tokens cleared

2.5 Logout

File: controllers/authController.js, services/authService.js

- Clears refresh token from MongoDB (set to null)
- Clears HTTP-only cookie from browser
- Requires valid access token (protected by verifyToken)
- POST /api/v1/auth/logout endpoint working

2.6 Production-Grade Architecture Upgrade

Major refactoring done on Day 3 — from basic to production-grade:

Service Layer Added (MVC+S Pattern)

- services/authService.js — all business logic moved here
- services/userService.js — user queries centralized
- Controllers now only handle req/res (thin layer)
- Clean separation: Routes → Controller → Service → Model

Input Validation — Zod

- utils/validators.js created with Zod schemas
- signupSchema: name (min 3), email (valid), password (min 8), role (enum)

- loginSchema: email (valid), password (min 8)
- .strict() mode — rejects unknown fields
- middleware/validateMiddleware.js — applied to routes
- Zod v4 uses .issues not .errors — fixed correctly

Security Middleware Stack

- helmet() — sets 15 security HTTP headers
- cors() — restricted to CLIENT_URL only
- express-rate-limit — auth routes: 10 req/15min, global: 100 req/15min
- xssClean — recursive XSS sanitization for body and query
- cookie-parser — for HTTP-only cookie handling

Error Handling

- utils/AppError.js — custom error class with statusCode
- middleware/errorMiddleware.js — centralized error handler
- Handles: MongoDB duplicate (11000), Zod errors, JWT errors
- All controllers use next(err) pattern

Logging — Winston

- utils/logger.js — structured JSON logging
- Logs to console + files (logs/error.log, logs/combined.log)
- Every request logged: METHOD URL - IP
- Login success/failure, signup, logout all logged
- logs/ folder added to .gitignore

Utility Files Created

File	Purpose
utils/generateToken.js	generateAccessToken (15m) + generateRefreshToken (7d)
utils/hashToken.js	bcrypt hash + compare for refresh tokens
utils/ApiResponse.js	Standard response format {success, message, data}
utils/AppError.js	Custom error class with statusCode property
utils/logger.js	Winston structured JSON logging
utils/validators.js	Zod validation schemas for signup and login

API Versioning

- All routes now use /api/v1/ prefix
- POST /api/v1/auth/signup
- POST /api/v1/auth/login
- POST /api/v1/auth/refresh
- POST /api/v1/auth/logout
- GET /api/v1/users/profile (protected)

- GET /api/v1/users/admin-only (admin RBAC)
- GET /health — health check endpoint

2.7 User Model Upgraded

File: models/User.js

- BCRYPT_SALT from environment variable (configurable)
- Email: lowercase, trim, regex validation added
- Password: minlength 8
- pre('save') hook — auto-hashes password before every save
- comparePassword() method — on user document
- toJSON() method — removes password from all API responses automatically
- refreshToken field added for refresh token storage
- versionKey: false — removes __v from responses

3. Complete Request Flow (Production Architecture)

Every API request goes through this pipeline:

- Client Request arrives
- Rate Limiter (10 req/15min for auth, 100 req/15min global)
- Helmet (security headers)
- CORS (restricted to CLIENT_URL only)
- express.json() + cookieParser()
- XSS Sanitizer (recursive — cleans all input fields)
- Request Logger (method, URL, IP logged by Winston)
- Routes (/api/v1/auth, /api/v1/users)
- Zod Validation Middleware (schema check)
- JWT Middleware — verifyToken (for protected routes)
- RBAC Middleware — authorizeRole (for role-restricted routes)
- Controller (only handles req/res)
- Service (all business logic here)
- Model (MongoDB operations)
- ApiResponse (standardized output format)
- Error Handler (catches all errors centrally)

4. API Test Results

Test Scenario	Status	Result
POST /api/v1/auth/signup (valid)	Done	201 — User registered successfully
POST /api/v1/auth/signup (weak password)	Done	400 — Password must be at least 8 characters
POST /api/v1/auth/signup (role: admin)	Done	400 — Zod rejects admin role
POST /api/v1/auth/signup (duplicate email)	Done	409 — Email already registered

POST /api/v1/auth/login (correct)	Done	200 — accessToken returned in body
POST /api/v1/auth/login (wrong password)	Done	401 — Invalid credentials
POST /api/v1/auth/login (missing field)	Done	400 — Validation failed
GET /api/v1/users/profile (no token)	Done	401 — No token provided
GET /api/v1/users/profile (valid token)	Done	200 — User info returned
GET /api/v1/users/admin-only (candidate)	Done	403 — Access denied
GET /api/v1/users/admin-only (admin)	Done	200 — Welcome Admin!
GET /health	Done	200 — status ok + timestamp

5. Mistakes Made & How Fixed

#	Mistake	Root Cause	Fix Applied
1	jwt required inside refresh function	Copy-paste without moving to top of file	Moved const jwt = require to top of file
2	Zod error used .errors not .issues	Zod v4 changed API structure from v3	Used error.issues.map() instead of error.errors.map()
3	mongoSanitize crashed server	Incompatible with newer Node.js req.query	Removed package — Zod already handles injection attacks
4	Hyperlink format in code from copy-paste	Copied from chat instead of typing manually	Always type code manually — never copy from chat interface
5	hashToken used without await in refresh	Async function called without await keyword	Added await before hashToken(newRefreshToken)

6. Key Concepts Learned Today

Service Layer Pattern (MVC+S)

Layer	File	Responsibility
Routes	routes/*.js	Define URL paths only — traffic director
Controller	controllers/*.js	Handle HTTP req/res and status codes only
Service	services/*.js	All business logic, DB queries, calculations
Model	models/User.js	MongoDB schema and model methods

Access Token vs Refresh Token

Feature	Access Token	Refresh Token
Expiry	15 minutes	7 days
Purpose	Used for API requests	Used to get new access token

Storage	Response body (frontend memory)	HTTP-only cookie
DB Storage	Not stored	Hashed with bcrypt before storing
Security	Short life = less risk	Hashed = safe even if DB hacked

HTTP-only Cookies

- Refresh token stored in HTTP-only cookie (not in response body)
- JavaScript CANNOT read HTTP-only cookies — XSS protection
- secure: true in production (HTTPS only)
- sameSite: Strict prevents CSRF attacks

Zod Validation

- safeParse() returns {success, data, error} without throwing
- parse() throws ZodError if validation fails
- .strict() rejects unknown fields not in schema
- error.issues (NOT error.errors) in Zod v4

Winston Logging

- Structured JSON logs with timestamp, level, message
- Three levels: info (normal), warn (suspicious), error (failure)
- Logs to console AND files simultaneously
- Production-grade logging is essential for debugging issues

7. New Files Created Today

File	Purpose
services/authService.js	signup, login, refresh, logout business logic
services/userService.js	getUsers, getUserById with secure field selection
middleware/validateMiddleware.js	Zod validation middleware applied to routes
middleware/xssMiddleware.js	Recursive XSS sanitization for all inputs
middleware/errorMiddleware.js	Centralized error handler for all routes
utils/generateToken.js	Access token (15m) + Refresh token (7d) generators
utils/hashToken.js	bcrypt hash and compare for refresh tokens
utils/ApiResponse.js	Standard response format class
utils/AppError.js	Custom error class with statusCode
utils/logger.js	Winston structured JSON logging
utils/validators.js	Zod schemas for signup and login
logs/error.log	Error logs (gitignored)

logs/combined.log	All logs combined (gitignored)
-------------------	--------------------------------

8. Files Modified Today

File	What Changed
models/User.js	Added pre-save hook, comparePassword, toJSON, refreshToken field
controllers/authController.js	Refactored to thin layer — uses authService
routes/authRoutes.js	Added refresh, logout routes and Zod validation
routes/userRoutes.js	Added profile and admin-only routes
server.js	Added helmet, CORS restriction, rate limiting, logging, API versioning
.env	Added JWT_REFRESH_SECRET, BCRYPT_SALT, CLIENT_URL

9. GitHub Commits Today

Commit Message	What Was Done
Day 3: Login API with JWT complete - Module 1 done	Login API + JWT tokens working
Day 3: RBAC complete - verifyToken and authorizeRole middleware	Protected routes + role check
Week 1: Production-grade auth system complete	Full security middleware stack added
Day 3: Fix Zod validation, remove mongoSanitize conflict	Bug fixes applied

10. EOD (End of Day) Summary

Task	Status
Login API implemented	Done
JWT Token Generation (access + refresh)	Done
Refresh token rotation system	Done
Logout API	Done
JWT verifyToken middleware	Done
Role-Based Access Control (RBAC)	Done
Service layer (MVC+S pattern)	Done
Zod input validation	Done
Helmet + CORS + Rate limiting	Done
XSS sanitization	Done
Winston logging	Done
Centralized error handling	Done

API versioning (/api/v1/)	Done
All 12 test scenarios passing	Done

11. Important Notes for Future Reference

- ALWAYS type code manually — never copy from chat (hyperlink format breaks code)
- async functions need await — always check every async call has await
- Zod v4 uses .issues not .errors — different from old tutorials
- mongoSanitize conflicts with newer Node.js — use Zod instead
- JWT_SECRET must be strong (32+ random characters)
- Never return password in API response — not even hashed version
- HTTP-only cookies protect refresh tokens from XSS attacks
- Rate limiting: auth routes max 10, global max 100 per 15 minutes
- API versioning /api/v1/ prefix for all routes — future-proof
- Winston logs in JSON format with timestamp — easier to search and filter
- Service layer is mandatory — controllers must NOT contain business logic

12. Tomorrow's Plan — Day 4

- React frontend setup using Vite (NOT Create React App — outdated)
- Install and configure Tailwind CSS
- Build Signup page component
- Build Login page component
- Connect frontend to backend API using axios
- Store JWT token securely (memory — not localStorage)
- Implement protected routes in React
- Test full flow in browser: Signup → Login → Dashboard
- Push to GitHub and send SOD + EOD
- Prepare for Friday weekly review with Sandiya